This policy sets a clear boundary. The agent can explore different strategies for completing its task, but every action must remain within the scope of the current customer.

Other policies restrict entire categories of actions. For instance, agents may be allowed to trigger workflows but not to modify them:

```
forbid (
    principal in Agent,
    action in [
        Action::"automation.create",
        Action::"automation.update",
        Action::"automation.delete"
    ],
    resource in Entity::Automation
);
```

This distinction between operational and control-plane actions is critical. Agents can carry out useful work, but they cannot alter the systems that define that work. In effect, the policy system “childproofs” the environment, enabling safe interaction while protecting critical controls.

18.3.3 From predefined workflows to governed behavior

The policy-aware loop marks a fundamental shift in how systems are designed. In traditional software, developers define workflows and insert authorization checks at predetermined points. The system’s behavior is predictable, and policies are written with that predictability in mind.

In agentic systems, behavior is not predefined. The agent determines its own sequence of actions based on its goals and the information it gathers. This makes static authorization models insufficient.

By integrating policy evaluation into the execution loop, ACME ensured that every action—regardless of how it was generated—would be assessed against the same set of rules. Policies no longer require knowing the application’s overall behavior in advance. Instead, they evaluate each action as it occurs.